



RentFinder Platform Architecture & System Design Document

Prepared by Picours Technology Company

Version 1.0 • Production Ready Architecture

Primary Currency: NAD (Namibian Dollar)

1. Architectural Goals

The platform architecture prioritizes financial safety, transparency, regional practicality, and long-term scalability. All payment operations are explicit, auditable, and resistant to accidental or malicious misuse.

2. High-Level System Architecture

The system consists of a role-based frontend, a stateless API server built with Node.js and Express, and a Supabase-backed PostgreSQL database enforcing financial invariants.

3. Component Responsibilities

Frontend applications manage presentation and user interaction only. The API server enforces business logic, state transitions, and validation. The database serves as the source of truth and enforces safety constraints.

4. Domain Model

Core domains include Tasks, Escrow, Payouts, Audit Logs, and Finance. Escrow acts as the mandatory gateway through which all money flows.

5. Escrow State Machine

pending → held → released | cancelled | disputed. Released and cancelled states are terminal. Disputed escrows require explicit admin resolution.

6. Financial Design

Each client payment of 250 NAD is split into 200 NAD for the finder and a 50 NAD non-refundable platform service fee. This split is enforced at database and application levels.

7. Payout Architecture

Escrow release creates a payout entitlement, not immediate money movement. Payouts are processed manually by admins to accommodate bank and mobile money constraints.

8. Admin Control Layer

Admins resolve disputes, process payouts, and monitor platform revenue. All admin actions are rate-limited, role-restricted, and logged.

9. Audit & Compliance

Every financial state change generates an immutable audit log recording actor, action, timestamp, and amount, supporting dispute resolution and regulatory needs.

10. Security Architecture

Security is enforced through layered defenses including JWT authentication, role-based access control, rate limiting, idempotent logic, and strict database constraints.

11. Scalability & Future Growth

The stateless API design and database-enforced invariants allow horizontal scaling. Future integrations such as automated payouts or new regions require no architectural rewrite.

12. Operational Resilience

The architecture is resilient to failure scenarios including payment errors, admin mistakes, and partial system outages, ensuring no loss or duplication of funds.

Conclusion

This architecture intentionally favors safety, transparency, and manual control. It is optimized for initial launch in Namibia and provides a strong foundation for future expansion.